

风险管理计划与应对计划

- 1、制定项目的风险管理计划；
- 2、分析项目的风险源、对风险进行定性和定量分析，给出风险的应对措施，最终形成项目的应对计划【要求，选择合适的方法和工具来进行描述】

1. 风险管理目标

提前识别、评估、控制影响进度、质量、功能交付的风险，保证关键模块按时完成，确保项目不延期，核心功能可用，稳定性达标。

2. 风险管理职责

马鑫：统筹风险管理全流程，主持风险评审会议，批准应对预算；TTS 模块技术风险排查与应对。

刘宇轩：前端界面及前后端集成风险排查。

梁艺馨：大模型接入与兼容性风险排查。

杨子琪：OCR 模块精度与性能风险排查。

王婧晋：屏幕捕获稳定性风险排查。

全体成员：日常风险观测，上报新风险，参与风险审查。

3. 风险管理方法与工具

风险识别：头脑风暴法、鱼骨图、核查表法。

定性分析：概率 - 影响矩阵、风险优先级评分。

定量分析：EMV 期望货币价值、决策树分析、蒙特卡洛模拟、敏感性分析。

应对规则：应对策略矩阵、应急响应计划。

风险监控：风险审查会议、偏差分析、风险再评估。

4. 风险管理安排

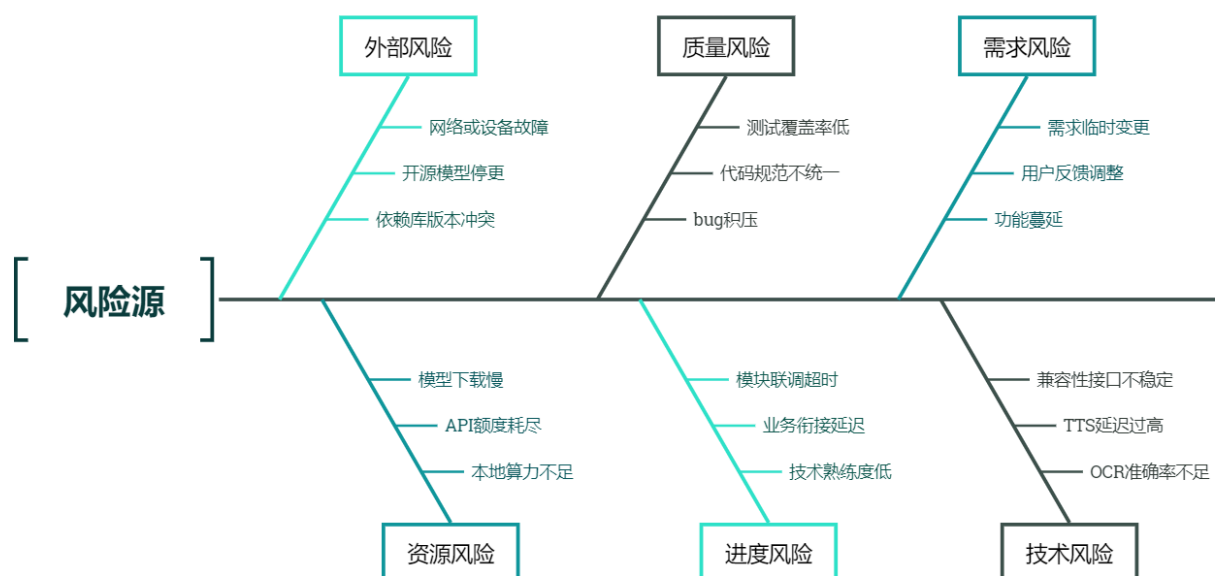
项目启动：初始风险识别与分析、应对计划制定、技术预研。

中期：定期开展风险审查。

结题：项目结束评审。

全程：持续风险监控。

5. 风险源分析

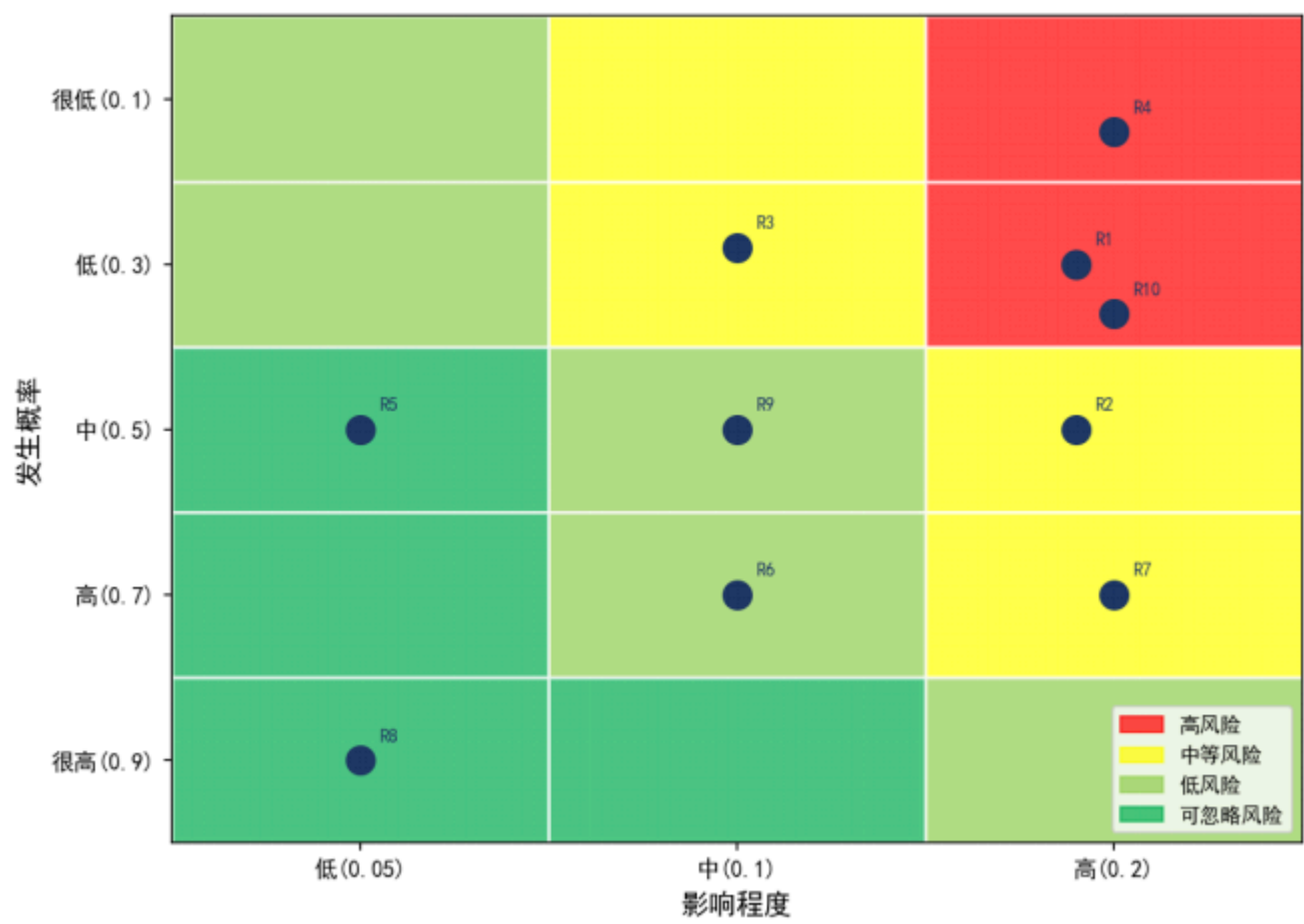


Presented with xmind

6. 风险登记册

风险 ID	风险名称	风险类别
R1	OCR 识别准确率不足	技术风险
R2	TTS 延迟过高	技术风险
R3	API 调用额度耗尽	资源风险
R4	AI 模型兼容性差	技术风险
R5	人员技术不熟练	进度风险
R6	需求临时变更	需求风险
R7	本地设备算力不足	资源风险
R8	数据隐私与安全风险	外部风险
R9	前端兼容性问题	技术风险
R10	多模块集成联调失败	技术风险

7. 风险概率 - 影响矩阵



8. 风险优先级排序

风险 ID	风险名称
R4	AI 模型兼容性差
R1	OCR 识别准确率不足
R10	多模块集成联调失败
R2	TTS 语音延迟过高
R3	API 调用额度耗尽
R7	本地设备算力不足
R9	前端兼容性问题
R5	人员技术不熟练
R6	需求临时变更
R8	数据隐私风险

9. 蒙特卡洛模拟

任务	乐观工期	最可能工期	悲观工期	期望工期
需求分析 A	2 天	3 天	4 天	3.0 天
系统设计 B	2 天	3 天	5 天	3.2 天
屏幕识别 C	6 天	8 天	12 天	8.3 天
文本转音频 D	6 天	8 天	11 天	8.2 天
系统集成 G	4 天	5 天	9 天	5.5 天
测试优化 H	3 天	4 天	6 天	4.2 天

10. 风险应对计划

风险 ID	风险名称	应对策略	具体措施	责任人
R4	AI 模型兼容性差	减轻、规避	① 项目第 3 天前完成所有拟用模型的安装验证 ② 建立备用模型清单 ③ 隔离各模型依赖环境 ④ 统一接口封装层，屏蔽底层差异	梁艺馨 马鑫
R1	OCR 识别准确率不足	减轻	① 采用多引擎方案 ② 图像预处理提升识别率 ③ 针对游戏场景设置 ROI 区域框选 ④ 定期准确率测试，低于 85% 立即调整预处理策略	杨子祺 王婧晋
R10	多模块集成联调失败	减轻	① 提前定义模块间接口规范 ② 开发阶段建立 Mock 接口，并行开发减少等待 ③ 集成前进行两两模块联调测试 ④ 集成期间每日代码合并，及时暴露冲突	马鑫 刘宇轩
R2	TTS 语音延迟过高	减轻、接受	① 优先使用轻量模型，降低推理延迟 ② 降低采样率 ③ 使用流式 TTS	马鑫
R3	API 调用额度耗尽	规避、转移	① 优先使用本地开源模型 ② 团队共享一个 API 账号，统一管理 token 使用	马鑫
R7	本地算力不足	转移、减轻	① 转移至性能最优的成员电脑作为主开发机 ② 轻量化模型替代 ③ 切换至 API 调用模式	杨子祺 王婧晋
R9	前端兼容性问题	减轻	① 统一开发环境 ② 定义 UI 分辨率适配规范 ③ 开发完成后在不同分辨率设备上测试 ④ 使用相对布局，避免绝对像素坐标	刘宇轩

R5	人员技术不熟练	减轻	① 项目开始前分享技术参考资料 ② 技术难点由项目经理优先攻关，形成 Demo 供参考 ③ 设置里程碑检查节点，及时发现进度偏差	马鑫
R6	需求临时变更	接受、减轻	① 变更需经组长审批，记录变更原因和影响 ② 保持最小可行版本，控制变更范围	全体
R8	数据隐私风险	减轻、接受	① 默认使用本地模型，避免屏幕内容上传云端 ② API 模式下对识别内容进行本地缓存，不长期存储 ③ 在用户手册中明确隐私说明	梁艺馨